김동욱

GraphQL에 기반한 Supergraph Architecture

클라우드 환경에서의 GraphQL

목차

01 GraphQL이란

02 GraphQL VS REST

03 Supergraph Architecture

04 아키텍처 구현

05 추가 고려 사항

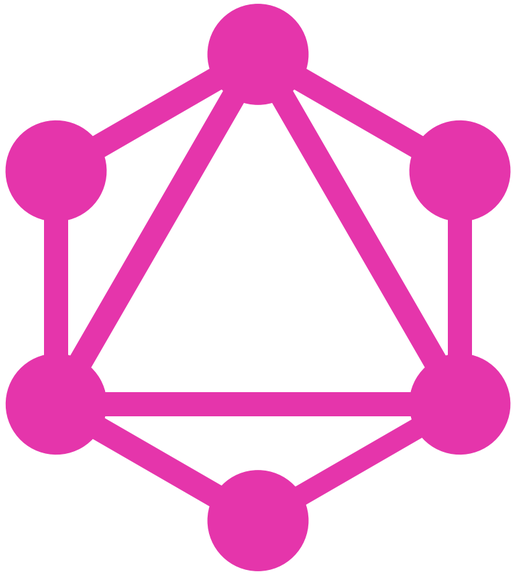
01

GraphQL이란

GraphQL 알아보기

01

GraphQL



01

GraphQL Schema

```
type Character {
  name: String!
  appearsIn: [Episode]!
}
```

Type

```
type Query {
  hero(episode: Episode): Character
}
```

Query

01

GraphQL Query

```
{
  hero {
    name
    appearsIn
  }
}
```

POST : /graphql

```
{
  "data": {
    "hero": {
      "name": "R2-D2",
      "appearsIn": [
        "NEWHOPE",
        "EMPIRE",
        "JEDI"
      ]
    }
  }
}
```

Query

Response

02

GraphQL VS REST

GraphQL과 REST의 차이

REST : Overfetching

```
{
  "name" : "Thonk",
  "profile" : "Thonk.png",
  "birth" : "2026-03-31",
  "address" : "Lorem Ipsum",
  "phone" : "000-0000-0000",
  "email" : "thonk@fisa.com",
  ...
}
```

사용할 데이터에 비해
받아야 할 내용이 너무 많다.

```
{
  "name" : "Thonk",
  "profile" : "Thonk.png"
}
```

GET : /user/{id}

Field in Actual Use

02

REST : Underfetching

```
{
  "name" : "Thonk",
  "profile" : "Thonk.png",
  ...
}
```

GET : /users/{id}

```
{
  "posts" : [
    "post1",
    ...
  ]
}
```

GET : /users/{id}/posts

```
{
  "comments" : [
    "comment1",
    ...
  ]
}
```

GET : /posts/{id}/comments

요청이 너무 많이 발생한다.

02

GraphQL

```
query defaultUserProfile($id: ID!) {
  user(id: $id) {
    name
    profile
  }
}

{
  "id" : "1"
}
```

```
query getUserWithPostsAndComments($id: ID!) {
  user(id: $id) {
    name
    profile
    posts {
      id
      title
      comments {
        id
        content
      }
    }
  }
}

{
  "id": "1"
}
```

POST : /graphql

02

GraphQL : Caching

요청 URL	https://api.github.com/users/torvalds
요청 메서드	GET
상태 코드	200 OK
원격 주소	20.200.245.245:443
리퍼러 정책	strict-origin-when-cross-origin

요청 URL	https://api.github.com/users/torvalds
요청 메서드	GET
상태 코드	304 Not Modified
원격 주소	20.200.245.245:443
리퍼러 정책	strict-origin-when-cross-origin

요청 URL	https://web.prod.cloud.netflix.com/graphql
요청 메서드	POST
상태 코드	200 OK
원격 주소	44.237.234.25:443
리퍼러 정책	strict-origin-when-cross-origin

무엇을 기준으로 캐싱하는가

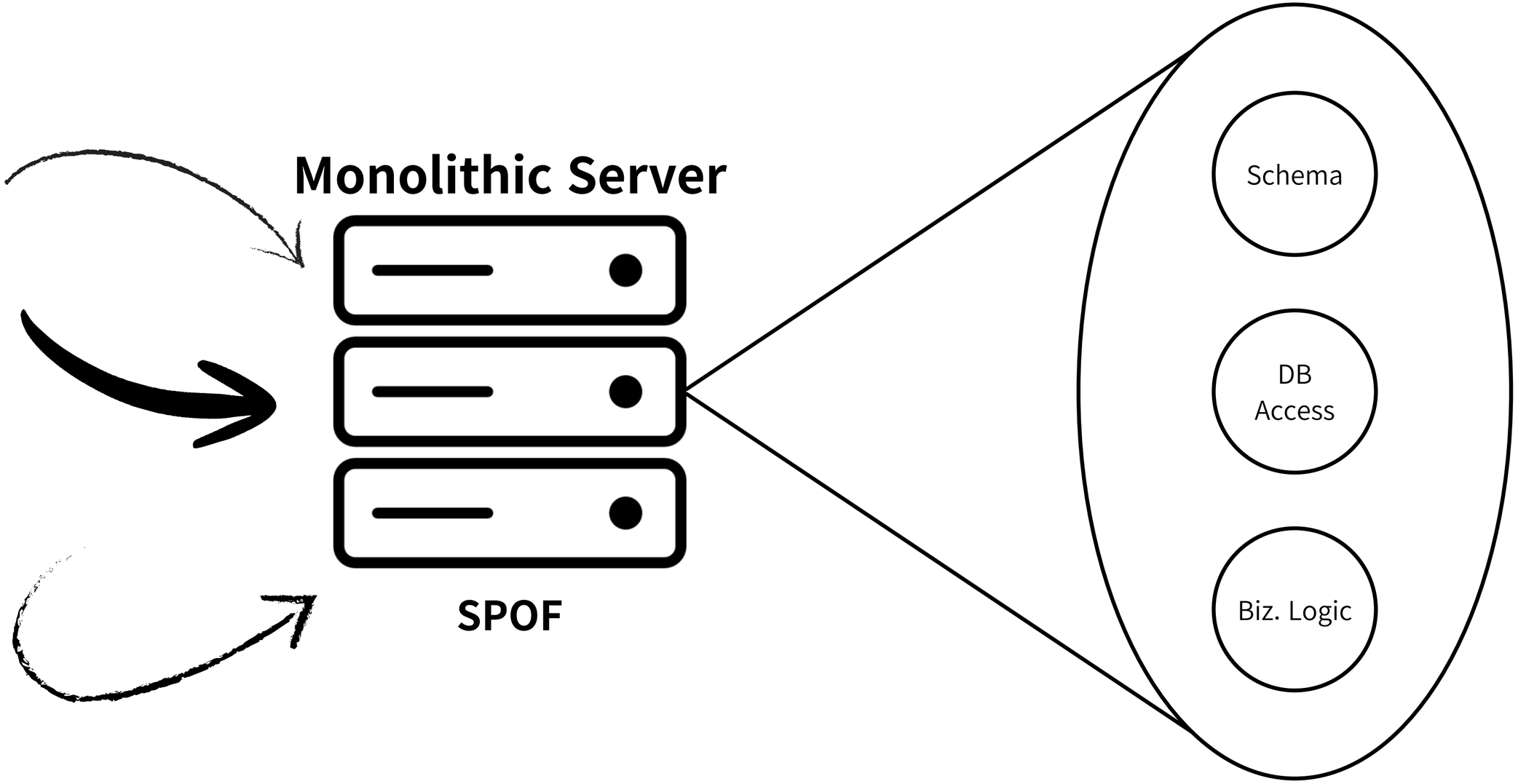
03

Supergraph Architecture

GraphQL의 Supergraph Architecture

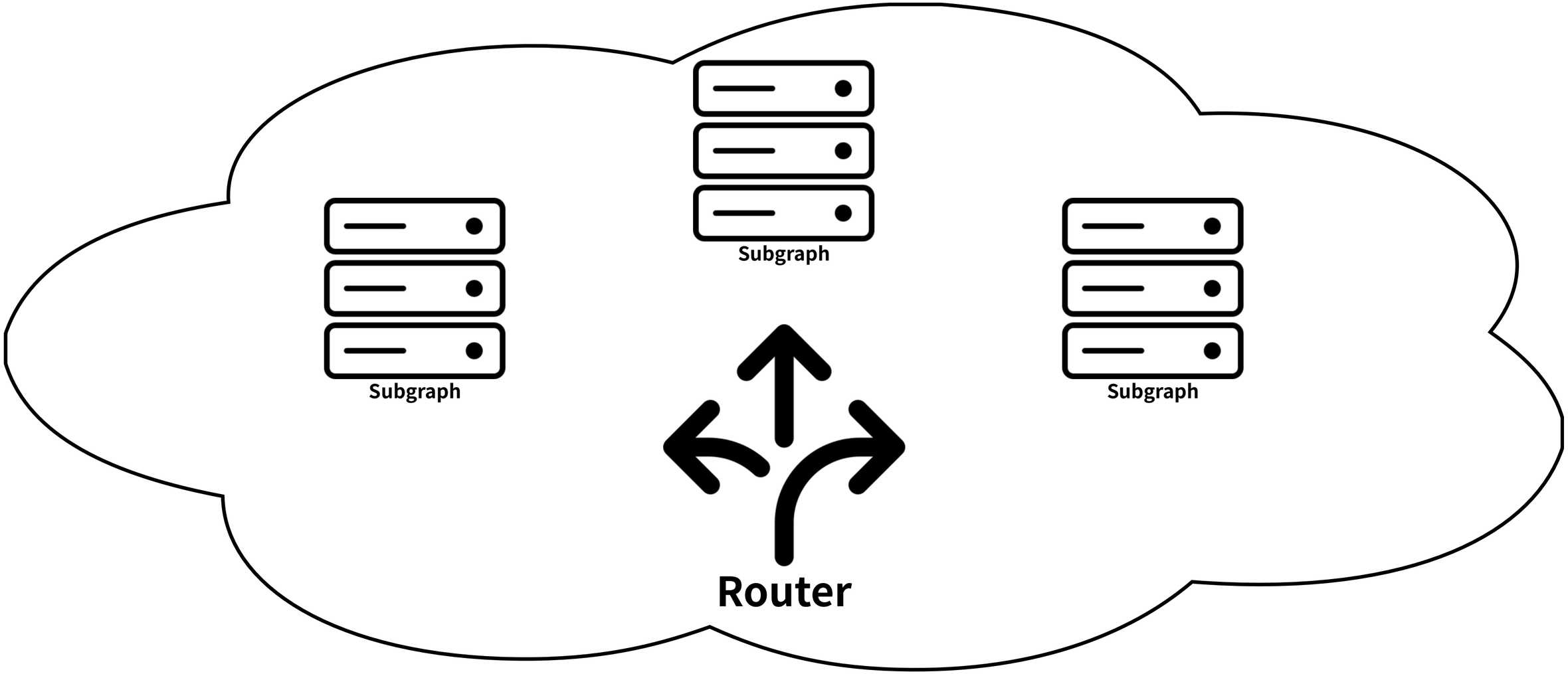
03

Monolithic Architecture



03

Supergraph Architecture



04

아키텍처 구현

Supergraph Architecture를 구현한다.

04

사용 기술

Netflix DGS

Spring Boot GraphQL Framework

Rover CLI

Subgraph Schema를 클라우드로 전송

Apollo GraphOS

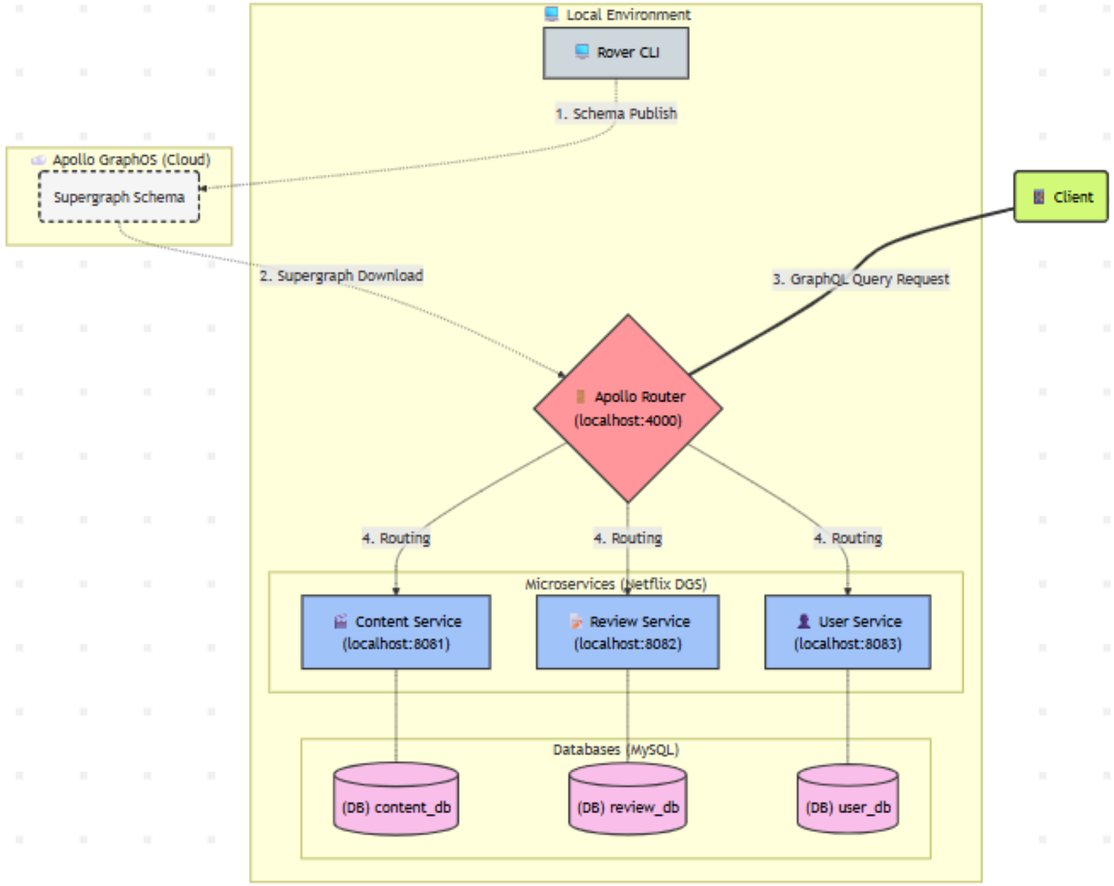
입력받은 Schema를 Supergraph화

Apollo Router

Supergraph를 기반으로 쿼리 라우팅

04

Entire Architecture



04

Schema

```
type Query {
    show(id: ID!): Show
    shows: [Show]
}

type Show @key(fields: "id") {
    id: ID!
    title: String!
    description: String
    releaseYear: Int
    previewVideoUrl: String
}
```

show

```
type Query {
    user(id: ID!): User
    users: [User]
}

type User @key(fields: "id") {
    id: ID!
    username: String!
    name: String!
    email: String!
}
```

user

```
type Query {
    review(id: ID!): Review
}

type Review {
    id: ID!
    rating: Int!
    comment: String
    user: User
    show: Show
}

type User @key(fields: "id") @extends {
    id: ID! @external
    reviews: [Review]
}

type Show @key(fields: "id") @extends {
    id: ID! @external
    reviews: [Review]
}
```

review

04

Netflix DGS

```
query {
  show(id: 1) {
    title
    reviews {
      rating
      comment
      user{
        name
      }
    }
  }
}
```

04

Netflix DGS

```
@DgsComponent   aransword
@RequiredArgsConstructor
public class ContentDataFetcher {
```

@DgsComponent

04

Netflix DGS

```
query {
  show(id: 1) {
    title
    reviews {
      rating
      comment
      user{
        name
      }
    }
  }
}
```

```
type Query {
    show(id: ID!): Show
    shows: [Show]
}

type Show @key(fields: "id") {
    id: ID!
    title: String!
    description: String
    releaseYear: Int
    previewVideoUrl: String
}
```

```
@DgsQuery(field = "show")  0개의 사용위치  aransword
public Show show(@InputArgument Long id) {
    return showRepository.findById(id).orElse( other: null);
}
```

04

Netflix DGS

```
query {
  show(id: 1) {
    title
    reviews {
      rating
      comment
      user{
        name
      }
    }
  }
}
```

```
type Show @key(fields: "id") @extends {
    id: ID! @external
    reviews: [Review]
}
```

```
@DgsEntityFetcher(name = "Show")  0개의 사용위치  신규 *
public Show resolveShow(Map<String, Object> values) {
    Long id = Long.parseLong(String.valueOf(values.get("id")));
    return new Show(id);
}

@DgsData(parentType = "Show", field = "reviews")  0개의 사용위치  aransword *
public List<Review> fetchReviewsForShow(DgsDataFetchingEnvironment dfe) {
    Show show = dfe.getSource();
    Long showId = show.getId();
    return reviewRepository.findByShowId(showId);
}
```

04

Netflix DGS

```
query {
  show(id: 1) {
    title
    reviews {
      rating
      comment
      user{
        name
      }
    }
  }
}
```

```
type Query {
    user(id: ID!): User
    users: [User]
}

type User @key(fields: "id") {
    id: ID!
    username: String!
    name: String!
    email: String!
}
```

```
@DgsData(parentType = "Review", field = "user")  0개의 사용위치  aransword
public Map<String, Object> fetchUserForReview(DgsDataFetchingEnvironment dfe) {
    Review review = dfe.getSource();
    return Map.of(k1: "id", review.getUserId().toString());
}
@DgsEntityFetcher(name = "User")  0개의 사용위치  aransword *
public User resolveUserReference(Map<String, Object> values) {
    String idStr = (String) values.get("id");
    if (idStr != null) {
        Long id = Long.parseLong(idStr);
        return userRepository.findById(id).orElse( other: null);
    }
    return null;
}
```

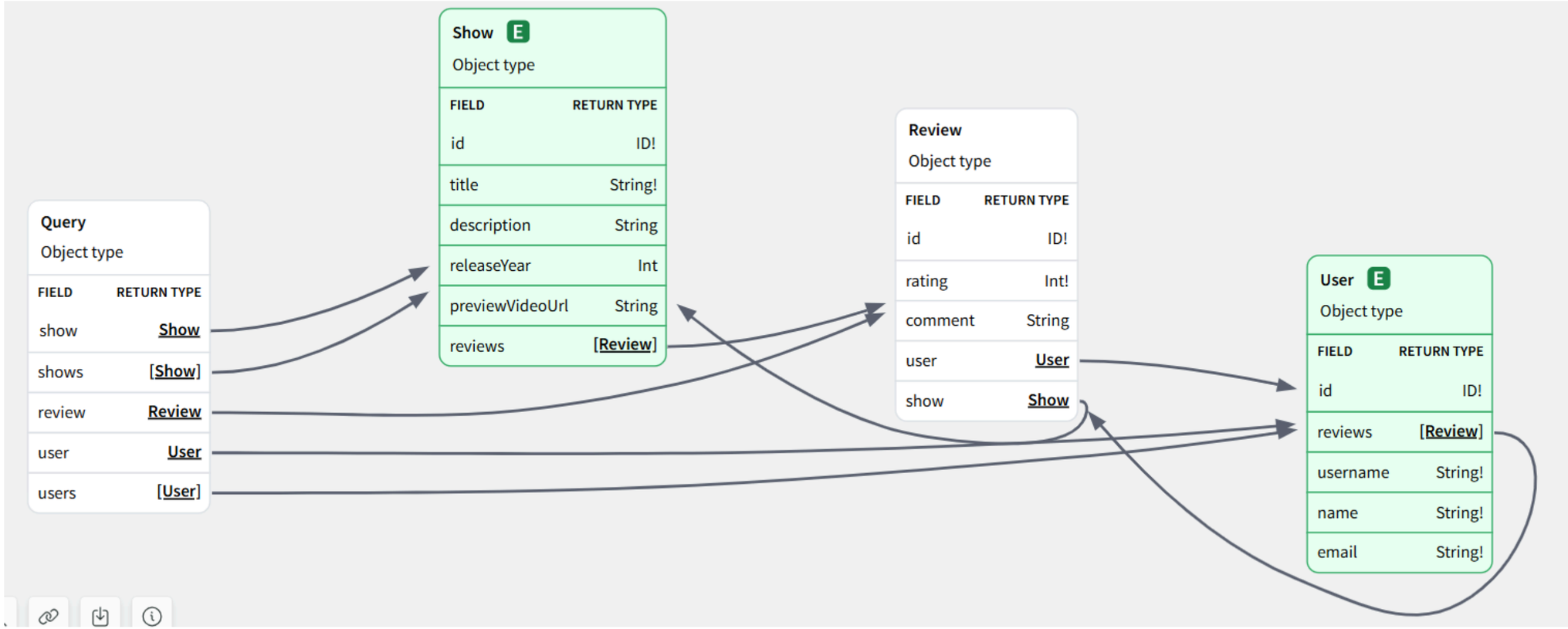
04

Rover CLI

```
$env:APOLLO_KEY="service:your-graph-name:YOUR_APOLLO_KEY"

rover subgraph publish your-graph-name@current --name users --schema
./user_service/src/main/resources/schema/schema.graphqls --routing-url http://localhost:8083/graphql

rover subgraph publish your-graph-name@current --name content --schema
./content_service/src/main/resources/schema/schema.graphqls --routing-url http://localhost:8081/graphql

rover subgraph publish your-graph-name@current --name reviews --schema
./review_service/src/main/resources/schema/schema.graphqls --routing-url http://localhost:8082/graphql
```

04

Apollo Studio



04

Apollo Router

```
$env:APOLLO_GRAPH_REF="Netflix-Graph@current"
.₩router.exe --dev
```

```
query {
  show(id: 1) {
    title
    reviews {
      rating
      comment
      user{
        name
      }
    }
  }
}
```

```
{
    "data": {
        "show": {
            "title": "기묘한 이야기",
            "reviews": [
                {
                    "rating": 5,
                    "comment": "최고의 시리즈입니다!",
                    "user": {
                        "name": "김제미니"
                    }
                },
                {
                    "rating": 3,
                    "comment": "시즌 1이 제일 재밌었어요.",
                    "user": {
                        "name": "박아폴로"
                    }
                },
                {
                    "rating": 5,
                    "comment": "일레븐 너무 귀엽고 스릴 넘쳐요!",
                    "user": {
                        "name": "피터 파커"
                    }
                }
            ]
        }
    }
}
```

POST : localhost:4000/

05

추가 고려 사항

GraphQL Federation 시 추가로 고려해야 할 사항

05

N+1 Problem

```
query {
  show(id: 1) {
    title
    reviews {
      rating
      comment
      user{
        name
      }
    }
  }
}
```

1

N : review 리스트의 개수만큼

05

N+1 Problem : DataLoader

```
SELECT name FROM USER WHERE id = 1;
SELECT name FROM USER WHERE id = 2;
SELECT name FROM USER WHERE id = 3;
SELECT name FROM USER WHERE id = 4;
--...
```

BATCHING

```
SELECT name FROM USER WHERE id IN (1, 2, 3, 4)
```

05

Caching

```
{
    "data": {
        "show": {
            "title": "기묘한 이야기",
            "reviews": [
                {
                    "rating": 5,
                    "comment": "최고의 시리즈입니다!",
                    "user": {
                        "name": "김제미니"
                    }
                },
                {
                    "rating": 3,
                    "comment": "시즌 1이 제일 재밌었어요.",
                    "user": {
                        "name": "박아폴로"
                    }
                },
                {
                    "rating": 5,
                    "comment": "일레븐 너무 귀엽고 스릴 넘쳐요!",
                    "user": {
                        "name": "피터 파커"
                    }
                }
            ]
        }
    }
}
```

캐싱의 기준이 무엇인가

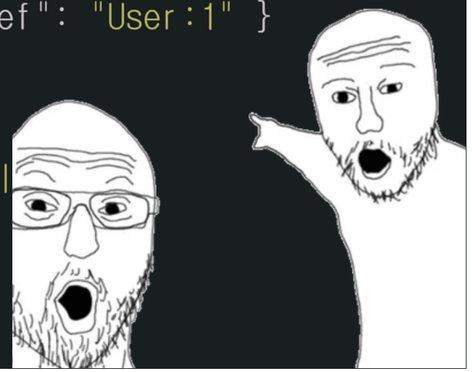
05

Caching : Apollo Client

```
{
  "show": {
    "__typename": "Show",
    "id": "1",
    "title": "기묘한 이야기",
    "reviews": [
      {
        "__typename": "Review",
        "id": "1",
        "rating": 5,
        "user": {
          "__typename": "User",
          "id": "1",
          "name": "김제미니"
        }
      }
    ]
  }
}
```

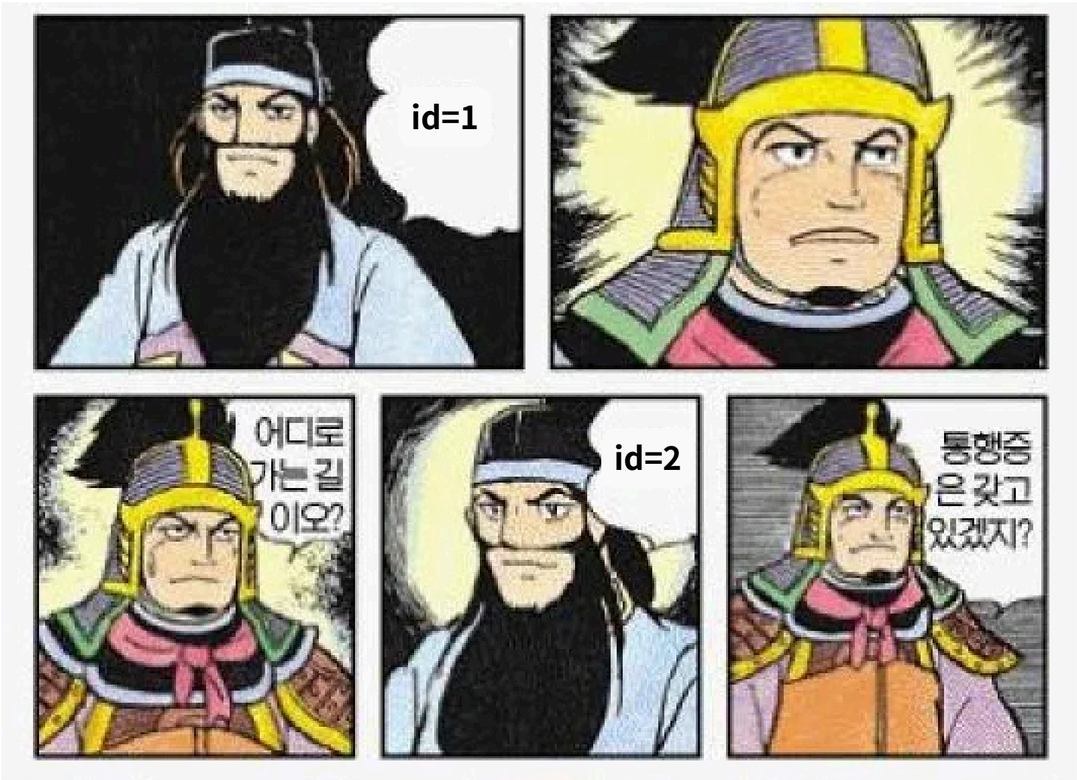
NORMALIZING

```
{
  "Show:1": {
    "id": "1",
    "title": "기묘한 이야기",
    "reviews": [ { "__ref": "Review:1" } ]
  },
  "Review:1": {
    "id": "1",
    "rating": 5,
    "user": { "__ref": "User:1" }
  },
  "User:1": {
    "id": "1",
    "name": "김제미
  }
}
```



05

IDOR



Fin.